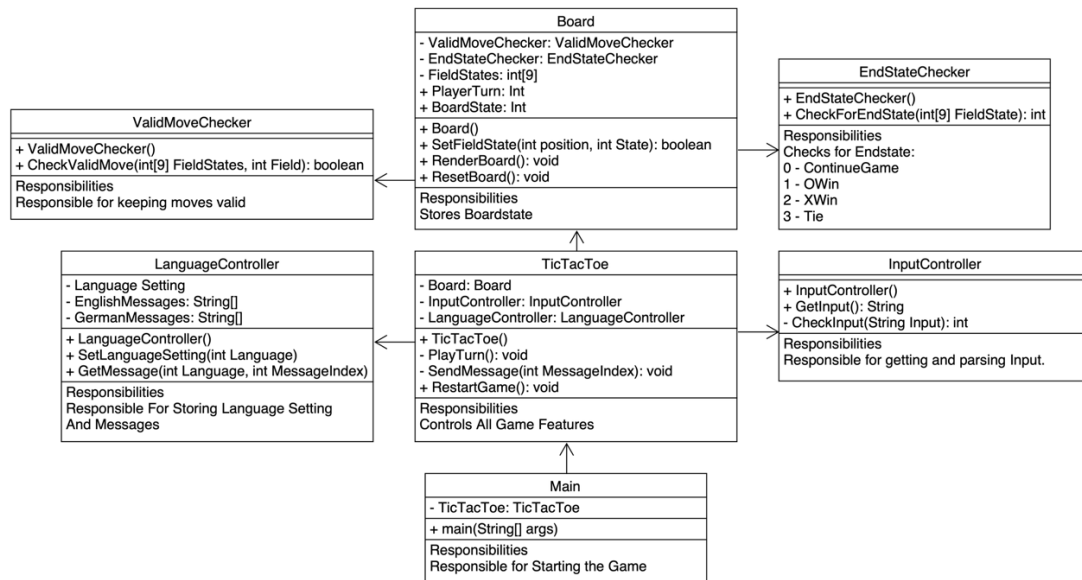


# Klassenbeschreibungen



## Main

**Rolle:** Startet das Spiel und initialisiert die Klasse **TicTacToe**.

**Attribute:**

- Referenz auf die Spielkontrollklasse **TicTacToe**.

**Methoden:**

- main(String[] args)** erstellt ein TicTacToe Objekt und startet das Spiel.

**Abhängigkeiten & Collabs:**

- Erzeugt **TicTacToe**.

## TicTacToe

**Rolle:** Kontrollzentrum aller Spielfunktionen.

**Attribute:**

- board: Board** repräsentiert den Spielzustand (die Felder, die Spieler und den BoardStatus)
- InputController: InputController** kümmert sich um die Eingaben der Spieler.
- LanguageController: LanguageController** gibt, je nach Sprache, Nachrichten an die Spieler aus.

**Methoden:**

- TicTacToe:** Konstruktor, der das Board, den Input- und LanguageController initialisiert.
- PlayTurn(): void** führt einen Spielzug aus: Eingabe, Validierung, Board Update, Endzustand.
- SendMessage(int MessageIndex): void** holt eine Nachricht beim LanguageController.

**Abhängigkeiten & Collabs:** xyz

- Verwendet **Board** für dessen Statusverwaltung und Rendering.
- Verwendet **InputController** zur Verarbeitung der Eingaben.
- Verwendet **LanguageController**, um Meldungen in der Console an die Spieler auszugeben.

## Board

**Rolle:** Speichert und kontrolliert den Spielzustand (die Felder, die aktuellen Spieler, den Board Status) und kann das Board rendern und zurücksetzen.

**Attribute:**

- **ValidMoveChecker** : **ValidMoveChecker** überprüft ob die Spielzüge gültig sind.
- **EndStateChecker** : **EndStateChecker** überprüft ob nach dem Zug ein Gewinn oder ein Unentschieden stattgefunden haben und setzt sonst das Spiel fort.
- **FieldStates: int[9]** ist der Zustand der 9 Spielfelder. (Beispiel: 1=leer, 2=O, 3=X)
- **PlayerTurn: int** gibt an, wer gerade am Zug ist.
- **BoardState: int** ist der Gesamtzustand des Boards.

**Methoden:**

- **Board()** initialisiert die Feldzustände und die Startspieler.
- **SetFieldState(int position, int state): boolean** validiert einen Zug, setzt das Feld und gibt true zurück, wenn alles geklappt hat.
- **RenderBoard(): void** gibt das Board in einer lesbaren Form in der Console aus.
- **ResetBoard(): void** setzt das Board auf den Anfangszustand zurück.

**Abhängigkeiten & Collabs:**

- Checkt mit **ValidMoveChecker** ob ein Spielzug valid ist.
- Checkt mit **EndStateChecker** den Status vom Board nach einem Zug und ob das Spiel weiter geht, jemand gewonnen, oder unentschieden ist.

## Input Controller

**Rolle:** Sorgt für die Spielereingaben und deren Parsing in Feldpositionen.

**Attribute:**

- -

**Methoden:**

- **InputController()** Konstruktor
- **GetInput(): String** liest die Spielereingabe in der Konsole.
- **CheckInput(String Input): int** überprüft und parst die Eingabe in eine gültige Feldposition.

**Abhängigkeiten & Collabs:**

- Wird durch **TicTacToe** aufgerufen

## ValidMoveChecker

**Rolle:** Stellt sicher, dass nur gültige Züge auf dem Board passieren, führt diese aber nicht durch -> Nur Überprüfung!

**Attribute:**

- -

**Methoden:**

- **ValidMoveChecker()** Konstruktor
- **CheckValidMove(int[9] FieldStates, int Field): boolean** überprüft, ob das angegebene Feld im gültigen Bereich liegt und aktuell leer ist.

**Abhängigkeiten & Collabs:**

- Wird durch **Board** verwendet, vor allem in **SetFieldState**.

## EdstateChecker

**Rolle:** Überprüft nach jedem Zug, ob das Spiel weitergeht, jemand gewonnen hat oder unentschieden ist.

**Attribute:**

- -

**Methoden:**

- **EndStateChecker()** Konstruktor
- **CheckForEndState(int[9] FieldState): int** gibt 0 (weiter), 1 (O gewinnt), 2 (X gewinnt), oder 3 (Unentschieden) zurück.

**Abhängigkeiten & Collabs:**

- Wird durch **Board** verwendet nach jedem gesetzten, validierten Zug.

## Language Controller

**Rolle:** Verwaltet die Spracheinstellungen und beinhaltet alle Textbausteine bzw. die Console-Nachrichten an die Spieler.

**Attribute:**

- **languageSetting: int** ist die aktuell gewählte Sprache (zB. 0 = Deutsch, 1 = Englisch)
- **englishMessages: String[]** alle Nachrichten auf Englisch.
- **germanMessages: String[]** alle Nachrichten auf Deutsch.

**Methoden:**

- **LanguageController()** Konstruktor, der die Default Sprache und die Nachrichtenarrays initialisiert.
- **SetLanguageSetting(int language)** setzt die aktuelle Sprache.
- **GetMessage(int language, int messageIndex)** liefert die richtige Nachricht an die Spieler.

**Abhängigkeiten & Collabs:**

- Wird durch **TicTacToe** für das Anzeigen der Nachrichten verwendet.

## TL:DR

1. **Main** startet alles
2. **TicTacToe** steuert alles
3. Spielzustand im **Board**
4. Eingaben über **InputController**
5. Überprüfung mit **ValidMoveChecker**
6. & **EndstateChecker**
7. Nachrichten an die Spieler mit **LanguageController**